FIXIT GHANA

Mobile App Product, Technology & UI/UX Specification

Customer + Professional service marketplace • MVP v1.0

Implementation reference for Claude Code

Flutter + Laravel • Ghana market

1. Product Overview

FixIt Ghana is a mobile marketplace connecting customers who need services with trusted local professionals. Customers post a job, professionals send quotations, customers compare offers and hire, then the job is tracked through completion and review.

Positioning: Find trusted professionals. Get the job done.

2. MVP Problem & Target

• Customers struggle to find trustworthy technicians quickly.

• Informal WhatsApp/social-media referrals make pricing and availability unclear.

• Professionals struggle to consistently find quality leads.

• FixIt creates a structured request → quote → hire → job → review workflow.

User	Primary goal	Core journey
Customer	Solve a service problem	Request → quotes → hire → track → review
Professional	Find and complete jobs	Verify → receive request → quote → work → earn
Admin	Operate the marketplace	Verify → monitor → moderate → report

3. MVP Service Categories

• Plumbing

• Electrical

• AC repair & servicing

• Cleaning

• Carpentry

• Painting

• Appliance repair

• Moving/furniture services

4. Recommended Tech Stack

Layer	Technology	Purpose
Mobile	Flutter + Dart	Android/iOS from one codebase
State	Riverpod	Scalable application state
Navigation	GoRouter	Role-aware navigation and route guards
Networking	Dio	REST API communication
Backend	Laravel	API, business rules and authorization
Auth	Laravel Sanctum	Mobile token authentication
Database	PostgreSQL	Relational production database
Notifications	Firebase Cloud Messaging	Job/quote/status alerts

Layer	Technology	Purpose
Maps	Google Maps Platform	Location selection and future matching
Storage	S3-compatible object storage	Photos and media
Payments	Paystack or Hubtel	Future Ghana payment processing
Admin	React + Tailwind CSS	Operations dashboard

5. How It Will Be Built

Architecture: **Flutter UI** → **Riverpod** → **Repository** → **Dio** → **Laravel /api/v1** → **Services/Policies** → **PostgreSQL**. FCM handles notifications, object storage handles media and Maps handles location features.

• Backend owns business rules and permissions.

• Use Laravel Policies/Gates for ownership.

• Use pagination and image compression.

• Keep secrets off the mobile app.

• Use shared Flutter design tokens and reusable components.

6. Navigation

Customer: Home • Requests • Messages • Activity • Profile

Professional: Home • Jobs • Messages • Earnings • Profile

7. App Colors & Visual System

Token	Hex	Use
Primary Green	#16A34A	Primary buttons, active states, positive actions
Primary Dark	#15803D	Pressed states and brand emphasis
Deep Green	#166534	High-contrast accents
Background	#F8FAFC	Main page background
Surface	#FFFFFF	Cards, sheets, inputs
Text	#0F172A	Primary text
Secondary	#64748B	Descriptions/metadata
Border	#E2E8F0	Dividers and input borders
Warning	#F59E0B	Pending/attention
Error	#DC2626	Errors/destructive actions
Info	#2563EB	Informational messages

Design Rules

• Use green strategically rather than everywhere.

• Use white surfaces over a very light background.

• Use 10–16 px corner radii.

• Use Inter or a similar modern sans-serif.

• Buttons generally 48–52 px high.

• Make verification, ratings and completed jobs highly visible.

• Avoid excessive gradients, shadows and crowded dashboards.

• Design for one-handed use and inconsistent mobile networks.

8. Customer UI/UX Screens

8.1 Splash

Purpose: Brand recognition and fast launch.

UI layout: Light background, centered FixIt logo and short tagline. No unnecessary long animation.

Interaction: Restore session and route to onboarding, login or role dashboard.

8.2 Onboarding 1 — Find Help

Purpose: Explain the core value.

UI layout: Large service illustration, heading, short description, page indicator and Continue button.

Interaction: Continue advances; optional Skip.

8.3 Onboarding 2 — Compare Quotes

Purpose: Explain the marketplace advantage.

UI layout: Illustration of professional cards with ratings and prices.

Interaction: Continue advances.

8.4 Onboarding 3 — Get It Done

Purpose: Explain tracking and reviews.

UI layout: Simple job timeline visual; Get Started and Login actions.

Interaction: Persist onboarding completion.

8.5 Login

Purpose: Authenticate a user.

UI layout: Welcome heading, phone/email, password, Forgot Password, Login, optional Google and Register link.

Interaction: Inline validation; loading state; route by role.

8.6 Registration

Purpose: Create an account.

UI layout: Name, phone/email, password, confirmation, terms checkbox and Create Account.

Interaction: Validate and continue to role selection.

8.7 Role Selection

Purpose: Choose Customer or Professional.

UI layout: Two large selectable cards with icons and descriptions.

Interaction: Selected card is highlighted; Continue activates.

8.8 Customer Home

Purpose: Start a request immediately.

UI layout: Greeting, large “What do you need help with?” action, popular services, active request and trust message.

Interaction: Category tap starts request.

8.9 Service Categories

Purpose: Browse services.

UI layout: Search plus 2/3-column icon grid with large icons and short labels.

Interaction: Selecting category opens request form.

8.10 Create Request — Details

Purpose: Describe the problem.

UI layout: Progress 1/3; description; up to 5 photo thumbnails; Continue.

Interaction: Photo upload progress/retry; preserve draft.

8.11 Create Request — Location

Purpose: Set service location.

UI layout: Progress 2/3; map/location card, locality/address and Change Location.

Interaction: Ask permission only when needed; allow manual entry.

8.12 Create Request — Schedule

Purpose: Set time and optional budget.

UI layout: Progress 3/3; ASAP/Today/Tomorrow/custom time; optional budget; Submit.

Interaction: Submit to backend and show success.

8.13 Request Submitted

Purpose: Reassure the customer.

UI layout: Success icon, request reference and View Request CTA.

Interaction: Open request or return Home.

8.14 Quotes

Purpose: Compare professionals.

UI layout: Cards show photo, verification, rating, jobs, quote price, availability and note.

Interaction: Tap profile or select quote.

8.15 Professional Profile

Purpose: Build trust before hiring.

UI layout: Photo, verified badge, rating, completed jobs, services, area, portfolio and reviews. Sticky Hire button.

Interaction: Hire opens confirmation.

8.16 Hire Confirmation

Purpose: Prevent accidental hiring.

UI layout: Summary sheet with professional, service, quote and time; Confirm Hire.

Interaction: Create job and notify professional.

8.17 Job Tracking

Purpose: Show progress.

UI layout: Job reference, professional summary and timeline: Confirmed → On the way → Arrived → Working → Completed.

Interaction: Message/call actions and status refresh.

8.18 Messages

Purpose: Support job communication.

UI layout: Job-context header, message bubbles, attachment and message field.

Interaction: Retry failed messages; show connection state.

8.19 Activity

Purpose: View request history.

UI layout: Active/Completed/Cancelled tabs with compact request cards.

Interaction: Tap opens detail.

8.20 Review

Purpose: Collect trust data.

UI layout: Stars plus Quality, Professionalism, Punctuality and optional comment.

Interaction: One review per completed job.

8.21 Profile

Purpose: Manage account.

UI layout: Avatar, identity information and grouped settings.

Interaction: Edit profile; secure logout.

9. Professional UI/UX Screens

9.1 Professional Onboarding

Purpose: Explain how professionals get customers.

UI layout: Three value cards: local leads, quotes, reputation; Become a Professional CTA.

Interaction: Start verification/profile setup.

9.2 Verification

Purpose: Establish trust.

UI layout: Step form for personal details, services, service area, experience, ID/document and profile photo.

Interaction: Save draft; submit for review.

9.3 Professional Home

Purpose: Operational overview.

UI layout: Greeting, verification state, new requests, today's jobs, earnings and quick actions.

Interaction: Open requests, jobs or earnings.

9.4 Job Requests

Purpose: Find suitable work.

UI layout: Cards show service, locality, distance, time, photos and budget; filters.

Interaction: Open request; decline or quote.

9.5 Request Detail

Purpose: Give enough information to price.

UI layout: Description, photos, location, time and budget if supplied.

Interaction: Send Quote CTA.

9.6 Send Quote

Purpose: Create an offer.

UI layout: Labour, materials estimate, total, availability and note.

Interaction: Validate and send quote.

9.7 Active Jobs

Purpose: Manage accepted work.

UI layout: Customer, service, time, status timeline and Update Status.

Interaction: Confirmed → On the way → Arrived → Working → Completed.

9.8 Earnings

Purpose: Show income.

UI layout: Available/pending/completed earnings, jobs and date filter.

Interaction: Future payout/withdrawal functionality.

9.9 Professional Profile

Purpose: Manage public reputation.

UI layout: Business name, verification, rating, jobs, services, areas, description, portfolio and reviews.

Interaction: Edit profile and portfolio.

10. Admin Dashboard

• Dashboard overview

• Customer management

• Professional verification

• Professional suspension/reactivation

• Job monitoring

• Dispute management

• Category management

• Review moderation

• Payments/commission reports

• Platform analytics

• Support tickets

• Admin roles and permissions

11. Database Core

Entity	Core fields
users	id, role, name, phone, email, password, status
professional_profiles	user_id, business_name, bio, experience, verification_status, rating
service_categories	id, name, icon, status
professional_services	professional_id, category_id, service_area
service_requests	id, customer_id, category_id, description, location, preferred_time, budget, status
request_images	id, request_id, path
quotes	id, request_id, professional_id, labour, materials, total, availability, note, status
jobs	id, request_id, quote_id, customer_id, professional_id, status, scheduled_at
messages	id, job_id, sender_id, body, attachment_path, read_at
reviews	id, job_id, customer_id, professional_id, rating, quality, professionalism, punctuality, comment
payments	id, job_id, amount, commission, provider, reference, status
notifications	id, user_id, type, title, body, read_at

12. API Structure

Base: /api/v1

• /auth/register, /auth/login, /auth/logout, /auth/me

• /services/categories

• /requests, /requests/{id}, /requests/{id}/quotes

• /quotes/{id}/accept

• /jobs, /jobs/{id}, /jobs/{id}/status

• /professionals/{id}

• /jobs/{id}/reviews

• /messages/{job}

• /professional/earnings

• /notifications

All protected endpoints require authentication plus role/ownership authorization.

13. Notifications

• New matching request

• New quote received

• Quote accepted

• Job status changed

• New message

• Job completed

• Review reminder

• Verification result

14. Payments & Monetization

For MVP, validate the marketplace loop before making payments a hard dependency. When ready, integrate a Ghana-friendly provider such as Paystack or Hubtel. Payment confirmation must be verified server-side.

• Commission on completed jobs.

• Professional Pro subscription for enhanced visibility/analytics.

• Clearly labelled featured professional placement.

• Future business plans for teams.

15. Matching Algorithm — MVP

• Match service category.

• Require active/verified professional status.

• Filter by service area.

• Check availability where possible.

• Rank by distance, rating, completed jobs and response rate.

• Distribute requests to a manageable set of relevant professionals.

Do not start with machine learning. Collect real marketplace data first.

16. Performance, Security & Reliability

• Paginate large lists and cache safe read data.

• Compress/cache images.

• Use skeleton loaders.

• Retry failed network requests safely.

• Never display mutation success before server confirmation.

• Use HTTPS and secure token storage.

• Use Laravel Policies/Gates.

• Rate-limit authentication and sensitive endpoints.

• Validate uploaded files.

• Never ship secret API keys in Flutter.

• Verify payment webhooks server-side.

17. Recommended Repository

/fixit
■■■ mobile/ — Flutter application
■■■ backend/ — Laravel REST API
■■■ admin/ — React + Tailwind dashboard
■■■ docs/ — product, UX, API, database and implementation documentation

18. Claude Code Build Contract

Treat this specification as the product/UX source of truth. Inspect the repository before modifying code.

• Build incrementally by feature/sprint.

• Create shared design tokens for the specified colors, typography, spacing and radii.

• Use Riverpod, GoRouter and repository-based API access.

• Keep business logic outside widgets.

• Use typed models and centralized error handling.

• Implement customer/professional route guards.

• Every network screen needs loading, empty, error and success states.

• Run formatter, analyzer/linter and tests after each feature.

• Do not use fake production responses.

• Do not modify unrelated working functionality.

• Do not claim completion until the feature is tested.

19. Development Phases

Phase	Deliverable
1	Flutter/Laravel/admin foundations + design system
2	Authentication + role selection
3	Customer request flow
4	Professional profile + verification
5	Matching + job feed + quotations
6	Hire flow + job lifecycle + messaging
7	Reviews + notifications + activity
8	Admin dashboard
9	Payments + commission
10	QA + security + performance + deployment

20. MVP Definition of Done

Customer: register → choose service → describe problem → upload photos → provide location/time → submit → receive quotes → inspect professional → hire → communicate → track → review.

Professional: register → submit verification → create profile → receive request → quote → get hired → update status → build reputation.

Admin: verify professionals and monitor users, requests, jobs and disputes.

21. Final Product Direction

FixIt should feel like a trusted modern service marketplace, not a classifieds app. The central action should always be obvious: **I have a problem** → **tell FixIt** → **compare trusted professionals** → **get it done.**

Build the reliable marketplace loop first. Add sophisticated payments, subscriptions, advanced matching and other growth features only after real usage validates them.